

Jenkins Practical Flow – A Bonsai Doc.

Initial run:

1. Redirect to [D:\Jenkins](#). This folder contains **jenkins.war**.
2. Open cmd and run **`java -jar jenkins.war`**.
3. This will give you a temporary admin password.
4. Install suggested plug-ins.
5. Create an admin account.
6. Install all of the listed plug-ins.

Manually installed plug-ins:

Docker:

Docker Pipeline

Docker Plugin

docker-build-step

Sonar:

SonarQube Scanner for Jenkins

MsBuild(Not necessary):

MSBuild Plugin

Graph:

Pipeline: Stage View Plugin.

Sequential runs:

1. Use **`java -jar jenkins.war`**
2. And use the credentials you have made.

Configurations

Tools:

1. JDK Installations:

Name

Default JDK

JAVA_HOME

C:\Program Files\Java\jdk-17

JDK installations

JDK installations ^ Edited

Add JDK

JDK

Name

Default JDK

JAVA_HOME

C:\Program Files\Java\jdk-17

☐ Install automatically ?

Add JDK

2. Git:

Name

Git

Path to Git executable

C:\Program Files\Git\bin\git.exe

Git installations

≡ Git

Name

Git

Path to Git executable ?

C:\Program Files\Git\bin\git.exe

☐ Install automatically ?

Add Git ▾

3. MSBuild Installations:

Name

MSBuilder

Default parameters

C:\Program Files\Microsoft Visual Studio\2022\Professional\MSBuild\Current\Bin

MSBuild installations

MSBuild installations ^

 Edited

Add MSBuild

MSBuild

Name

MSBuilder

Default parameters ?

C:\Program Files\Microsoft Visual Studio\2022\Professional\MSBuild\Current\Bin

☒ Install automatically ?

Install from Microsoft

Version ?

Build Tools 2022 

Additional Arguments ?

4. SonarQube Scanner:

Name

MySonar

SONAR_RUNNER_HOME

D:\sonarqube-25.1.0.102122\sonarqube-25.1.0.102122

SonarQube Scanner installations

SonarQube Scanner installations ^ Edited

Add SonarQube Scanner

☰ SonarQube Scanner

Name

MySonar

SONAR_RUNNER_HOME

D:\sonarqube-25.1.0.102122\sonarqube-25.1.0.102122

☐ Install automatically ?

Add SonarQube Scanner

5. Docker:

Name

Docker

Installation root

C:\Program Files\Docker\Docker

Docker installations

Docker installations ^

Edited

Add Docker

≡ Docker

Name

Docker

Installation root ?

C:\Program Files\Docker\Docker

☐ Install automatically ?

Add Docker

System:

Set during the initial run:

Jenkins Location

Jenkins URL [?](#)

http://localhost:8306/

 Please set a valid host name, instead of localhost

System Admin e-mail address [?](#)

address not configured yet <nobody@nowhere>

SonarQube Servers:

Name

Sonar URL

Server URL

<http://localhost:9000>

Server authentication token

- none -

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

Sonar URL

Server URL
Default is http://localhost:9000

http://localhost:9000

Server authentication token
SonarQube authentication token. Mandatory when anonymous access is disabled.

- none -

+ Add

Advanced

Add SonarQube

Declarative Pipeline (Docker):

Docker Label

MyDocker

Docker registry URL

tcp://localhost:2375

Registry credentials

- none -

Declarative Pipeline (Docker)

Docker Label ?

MyDocker

Docker registry URL ?

tcp://localhost2375

Registry credentials

- none -

+ Add

Practical Flow:

1. Create or have any project you already have with the following requirements:
 1. One fully functional project: Console App, Web API or MVC Razor Page.
 2. One Nunit template-like testing project.
 3. DockerFile for the main project alone.
2. Push code into a public repo.
3. Create a pipeline job in jenkins.
4. Write the pipeline script.

Full Script:

```
node {
  stage("Git Checkout") {
    git
    "https://github.com/BonsaiPlays/JenkinsTrainerFantasyWebApp.git"
  }
  stage("Build Project") {
    bat "dotnet build FantasyWebApp/FantasyWebApp.sln"
  }
  stage("Test Project") {
    bat "dotnet test FantasyWebApp/FantasyWebApp.sln"
  }
  stage("QA") {
    bat 'dotnet sonarscanner begin /k:"FantasyWebApp"
/d:sonar.host.url="http://localhost:9000"
/d:sonar.token="sqp_f366fa66c75dca587a266b0d0e51d1a370b9a474"'
    bat "dotnet build FantasyWebApp/FantasyWebApp.sln"
    bat 'dotnet sonarscanner end
/d:sonar.token="sqp_f366fa66c75dca587a266b0d0e51d1a370b9a474"'
  }
  stage("Docker Containerization") {
    dir("FantasyWebApp/FantasyWebApp") {
      bat ""
      for /f %%i in ('docker ps -a -q --filter "ancestor=pipeline"') do
docker rm -f %%i
      docker rmi -f pipeline
      docker build -t pipeline .
      docker run -d -p 8083:8083 --name pipelinecontainer pipeline
      ""
    }
  }
}
```

Pipeline Script Line by Line:

1. Pull latest code from your repoo:

```
stage("Git Checkout") {  
    git  
    "https://github.com/BonsaiPlays/JenkinsTrainerFantasyWebApp.git"  
}
```

2. Directly build your project:

```
stage("Build Project") {  
    bat "dotnet build FantasyWebApp/FantasyWebApp.sln"  
}
```

3. Test your project with the Nunit Tests:

```
stage("Test Project") {  
    bat "dotnet test FantasyWebApp/FantasyWebApp.sln"  
}
```

4. Run SonarQube and have the code analysed.

Have the regular run of SonarQube done to get the commands.
Have the commands in the single quotes.

```
stage("QA") {  
    bat 'dotnet sonarscanner begin /k:"FantasyWebApp"  
/d:sonar.host.url="http://localhost:9000"  
/d:sonar.token="sqp_f366fa66c75dca587a266b0d0e51d1a370b9a474"  
    bat "dotnet build FantasyWebApp/FantasyWebApp.sln"  
    bat 'dotnet sonarscanner end  
/d:sonar.token="sqp_f366fa66c75dca587a266b0d0e51d1a370b9a474"  
}
```

5. Docker Stage:

```
stage("Docker Containerization") {  
    dir("FantasyWebApp/FantasyWebApp") {  
        bat ""  
        for /f %%i in ('docker ps -a -q --filter "ancestor=pipeline"') do  
docker rm -f %%i  
        docker rmi -f pipeline  
        docker build -t pipeline .  
        docker run -d -p 5306:5306 --name pipelinecontainer pipeline  
        ""  
    }  
}
```

dir("FantasyWebApp/FantasyWebApp")

Redirects into the project file, I need to use it twice.

Most for most people using once is working fine: **dir("FantasyWebApp")**

for /f %%i in ('docker ps -a -q --filter "ancestor=<imageName>"') do
docker rm -f %%i

Similar to a for-each loop in the windows batch scripting, goes through all the containers to filter out container made using a particular image file and delete them one after the other.

docker rmi -f <imageName>

Delete the image file.

docker build -t <imageName> .

Create new image.

docker run -d -p <port>:<port> --name <containerName> <imageName>

Create and run a container based on the image created. The port must match the port mentioned in the docker file.

bat `` ... ``

Allows you to run multiple bat commands in sequence.

Docker File:

```
# Build stage

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

WORKDIR /src

# Copy the project file and restore dependencies
COPY FantasyWebApp.csproj ./

RUN dotnet restore

# Copy the rest of the source code
COPY . ./

RUN dotnet publish -c Release -o /app/publish

# Runtime stage

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime

WORKDIR /app

# Copy published output from build stage
COPY --from=build /app/publish .

# Expose port 5000 for the API
EXPOSE 5306

# Set environment variable for ASP.NET Core to listen on port 5000
ENV ASPNETCORE_URLS=http://+:5306

# Run the application
ENTRYPOINT ["dotnet", "FantasyWebApp.dll"]
```